Project Overview

Build a cross-platform mobile app (iOS & Android) where users can send media (photo, video, GIF) and, upon opening, the recipient's camera automatically records their reaction and uploads it back. All interactions are saved and tied to user accounts.

Updated Wireframes

*Note: The ASCII sketches below are **conceptual wireframes** to illustrate layout and user flow—they are **not** the final UI design. The real app will use modern styling, colors, animations, and a fresh, youthful look. Use a design tool like Figma or Sketch to create polished mockups based on these layouts.*

Below are rough ASCII sketches to illustrate key screens and their layouts. Use these as guides when you move to a design tool like Figma.

---

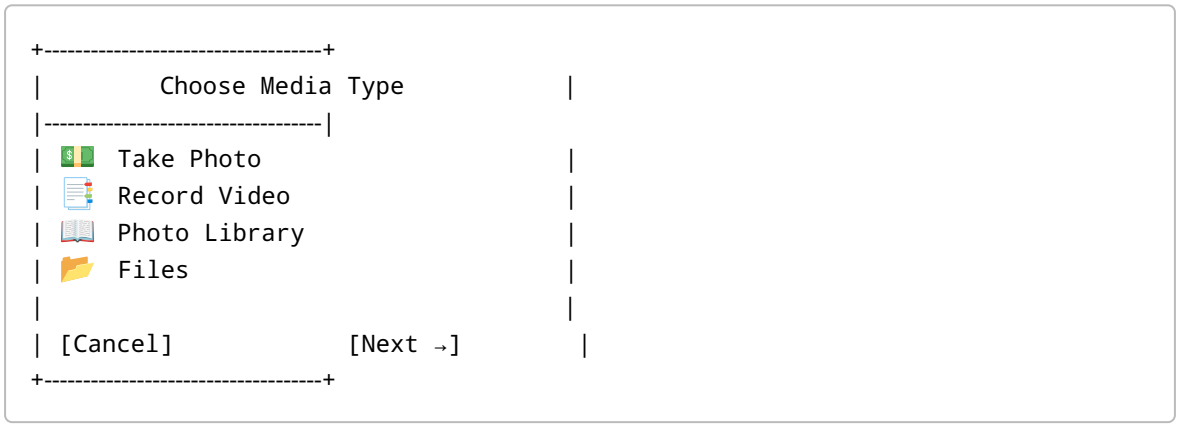
1. Onboarding / Permissions

```
+---------------------------------+
|   [App Logo]                    |
|                                 |
|  Welcome to ReactReact!         |
|  We need access to your camera  |
|  and microphone to record your  |
|  reactions.                     |
|                                 |
|  [ Allow Camera & Microphone ]  |
|  [ Exit App ]                   |
+---------------------------------+
```

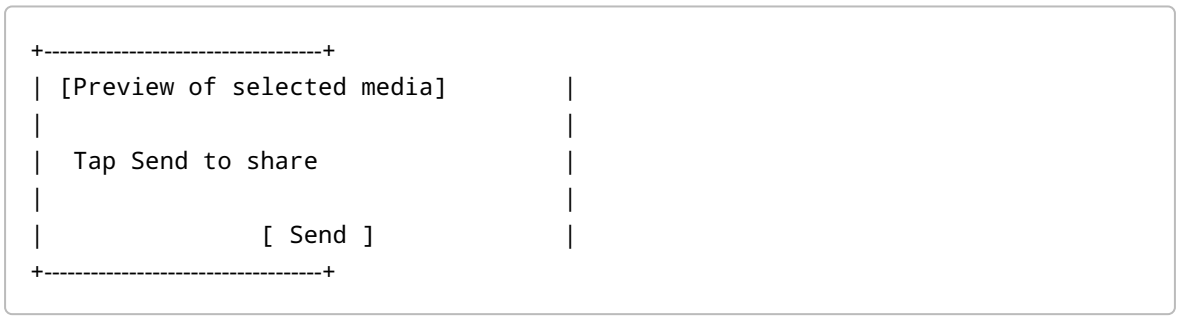
2. Inbox / Feed

```
+---------------------------------+
| 🔍 Inbox       [Profile Icon]    |
|---------------------------------|
|    PhotoFromAlice   2m ago      |
| 🔎 VideoFromBob     5m ago      |
| 🕛 GifFromCarol     1h ago      |
|                                 |
|     [ + Send New Media ]        |
+---------------------------------+
```

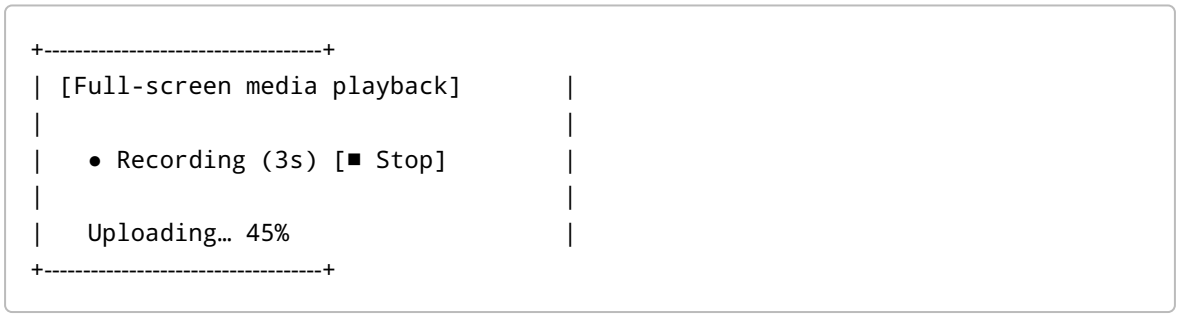
3. Send Media Options



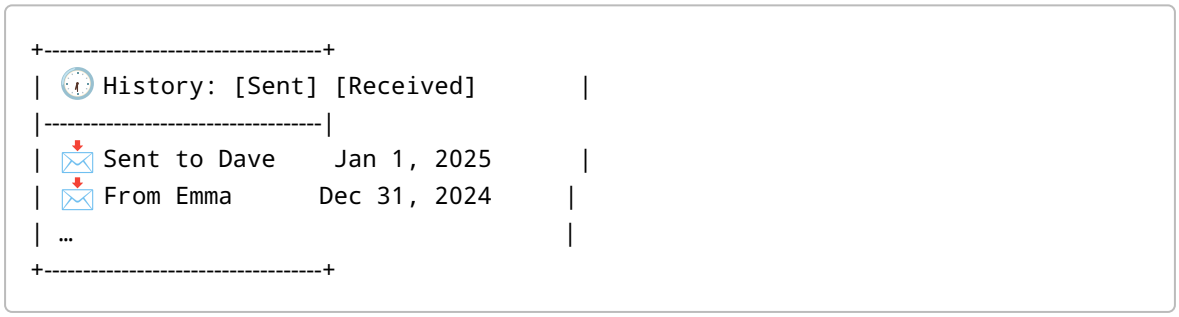
4. Media Preview & Send



5. View Message & Auto-Record



6. History



Next Steps

1. **Integrate Backend**\ Wire your mobile app to the FastAPI API: update `src/api.js` to point at your server's LAN IP, implement JWT storage (e.g. with AsyncStorage), and test signup/login flows end-to-end.

2. **Visual Design Polish**\ Move these layouts into a design tool (Figma, Sketch). Define a modern style guide—color palette, typography, iconography—and leverage a React Native UI library (e.g., React Native Paper or NativeBase) for consistent, high-quality components.

3. **Prototype & Feedback**\ Build clickable prototypes to refine navigation, animations, and micro-interactions before full implementation.

Let me know if these sketches match your vision or if you'd like any changes. Remember, these are just rough layouts—the final UI will be modern, engaging, and polished.

Step 5: Remove Expo Router & Enforce Custom Entry

Expo's file-based **Router** is still loading—let's remove **all** traces of it and force your own entry point.

Do these steps one by one, verifying each succeeds, before moving on:

5.1 Ensure the router plugin is installed

```
cd ~/projects/mobile-frontend
# Install expo-router to satisfy Expo Config Plugin
yarn add expo-router
```

5.2 Delete Router files & folders Delete Router files & folders

```
# Remove the app/ directory that contains router routes
rm -rf app
# Remove any Expo-shared router config
rm -rf .expo-shared
```

5.3 Remove router plugin from Babel (Bash)

```
# In project root, delete expo-router babel plugin line:
sed -i "/expo-router\/babel/d" babel.config.js
# Confirm removal:
grep -R "expo-router/babel" babel.config.js || echo "Removed";
```

5.4 Remove router plugin from app.json (Bash)

```
# Remove any expo-router plugin entries:
sed -i "/expo-router/d" app.json
# Inject entryPoint under the expo key:
sed -i "/\"expo\": *{/a \    \"entryPoint\": \"index.js\"," app.json
# Confirm entryPoint is set:
grep -R "entryPoint" app.json
```

5.5 Override the entry in package.json (Bash)

```
# If main exists, replace it; otherwise insert it after name:
if grep -q \"main\" package.json; then
 sed -i "s/\"main\": *\".*\"/\"main\": \"index.js\"/" package.json
else
 sed -i "/\"name\":/a \  \"main\": \"index.js\"," package.json
fi
# Confirm the new main entry:
grep -R "\"main\": \"index.js\"" package.json
```

5.6 Create the root index.js

```
cat > index.js << 'EOF'
import 'react-native-gesture-handler';
import { registerRootComponent } from 'expo';
import App from './App';

// Register App as the root component
registerRootComponent(App);
EOF
```

5.7 Clear caches & restart Expo (Bash)

```
rm -rf .expo .expo-shared node_modules yarn.lock
yarn install
npx expo start -c
``` Create the root index.js

```bash
cat > index.js << 'EOF'
import 'react-native-gesture-handler';
import { registerRootComponent } from 'expo';
import App from './App';

// Register App as the root component
```

```
registerRootComponent(App);
EOF
```

5.7 Clear caches & restart Expo Clear caches & restart Expo

```
# Delete any remaining cache folders
rm -rf .expo .expo-shared node_modules
# Install all dependencies fresh
yarn install
# Start Metro with cleared cache
yarn expo start --clear
```

> **Note:** On Windows, you may need `npx expo start -c`.

5.8 Test on device

1. In **Expo DevTools** switch **Connection** mode to **Tunnel**.
2. Scan the QR code in **Expo Go** on your phone.
3. You should now land on your **Login** screen (not “Unmatched Route”).

Step 6: Integrate Backend (Bash-only)

6.0 Create FastAPI Entry Point

```
cd ~/projects/backend-api
# Create main.py with a minimal FastAPI app if it doesn't exist
test -f main.py || cat > main.py << 'EOF'
from fastapi import FastAPI
app = FastAPI()
EOF
```

6.1 Configure Local Database (.env)

```
cd ~/projects/backend-api
# Use SQLite for quick setup
echo "DATABASE_URL=sqlite:///./test.db" > .env
```

6.2 Create (or replace) `database.py` with Conditional Engine

```
cd ~/projects/backend-api
# Remove any existing database.py to avoid stale code
rm -f database.py
cat > database.py << 'EOF'
from sqlalchemy import create_engine
```

```
from sqlalchemy.orm import sessionmaker
from dotenv import load_dotenv
from os import getenv
from models import Base

load_dotenv()
DATABASE_URL = getenv('DATABASE_URL')
if DATABASE_URL and DATABASE_URL.startswith('sqlite'):
    engine = create_engine(DATABASE_URL, connect_args={"check_same_thread":
False})
else:
    engine = create_engine(DATABASE_URL)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
# Create tables if they don't exist
Base.metadata.create_all(bind=engine)
EOF
```

touch **init**.py

If you see a psycopg2.OperationalError: invalid connection option "check_same_thread", delete and recreate `` with this code, then restart from Step 6.3.

6.3 Create Routers (auth.py & messages.py) Create Routers (auth.py & messages.py)

```
cd ~/projects/backend-api
# auth.py
cat > auth.py << 'EOF'
from fastapi import APIRouter, HTTPException
from passlib.context import CryptContext
from sqlalchemy.orm import Session
from database import SessionLocal
from models import User
import jwt, os

router = APIRouter()
pwd = CryptContext(schemes=["bcrypt"])

@router.post('/signup')
def signup(email: str, password: str):
    db = SessionLocal()
    if db.query(User).filter_by(email=email).first():
        raise HTTPException(400, "Email already registered")
    user = User(email=email, password_hash=pwd.hash(password))
    db.add(user); db.commit()
    token = jwt.encode({'user_id': user.id},
os.getenv('JWT_SECRET','secret'), algorithm='HS256')
    return {'access_token': token}

@router.post('/login')
```

```
def login(email: str, password: str):
    db = SessionLocal()
    user = db.query(User).filter_by(email=email).first()
    if not user or not pwd.verify(password, user.password_hash):
        raise HTTPException(400, "Invalid credentials")
    token = jwt.encode({'user_id': user.id},
os.getenv('JWT_SECRET','secret'), algorithm='HS256')
    return {'access_token': token}
EOF

# messages.py placeholder
cat > messages.py << 'EOF'
from fastapi import APIRouter
router = APIRouter()
# TODO: implement /messages/send and /messages/inbox
EOF
```

6.4 Register Routers in main.py

```
cd ~/projects/backend-api
grep -q "include_router(auth)" main.py || cat >> main.py << 'EOF'
from auth import router as auth_router
from messages import router as message_router
app.include_router(auth_router)
app.include_router(message_router)
EOF
```

6.5 Install Backend Dependencies

```
cd ~/projects/backend-api
source venv/Scripts/activate # Windows
pip install fastapi uvicorn[standard] sqlalchemy passlib[bcrypt] python-dotenv
PyJWT
```

6.6 Run FastAPI Server on LAN

```
cd ~/projects/backend-api
python -m uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

6.7 Update Mobile App baseURL

```
cd ~/projects/mobile-frontend
sed -i "s|127.0.0.1|<YOUR_PC_IP>|g" src/api.js
```

6.8 Install AsyncStorage

```
cd ~/projects/mobile-frontend
yarn add @react-native-async-storage/async-storage
```

6.9 Overwrite LoginScreen to Store Token

```
cd ~/projects/mobile-frontend
cat > src/screens/LoginScreen.js << 'EOF'
import React, {useState} from 'react';
import {View, TextInput, Button, Alert} from 'react-native';
import api from '../api';
import AsyncStorage from '@react-native-async-storage/async-storage';
export default function LoginScreen({navigation}) {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const handleLogin = async () => {
    try {
      const res = await api.post('/signup', { email, password });
      await AsyncStorage.setItem('token', res.data.access_token);
      navigation.replace('Inbox');
    } catch {
      Alert.alert('Error','Login/signup failed');
    }
  };
  return (
    <View style={{flex:1,justifyContent:'center',padding:16}}>
      <TextInput placeholder="Email" value={email} onChangeText={setEmail} />
      <TextInput placeholder="Password" secureTextEntry value={password}
onChangeText={setPassword} />
      <Button title="Login / Signup" onPress={handleLogin} />
    </View>
  );
}
EOF
```

6.10 Restart Expo & Test

```
cd ~/projects/mobile-frontend
npx expo start -c
```